Ketaki Dabade
kvd2112@columbia.edu
LinkedIn | GitHub | Portfolio

PaperTrail - SEC Filing Contradiction Detector

What I Built

PaperTrail is an agentic AI system that monitors SEC filings for S&P 500 companies and automatically detects when a company contradicts something it said in a previous filing. The idea came from a simple observation: public companies file thousands of pages per quarter, and nobody has time to cross-reference what Costco said about their accounting policies in March against what they said in January. An analyst might catch it if they happen to remember, but that doesn't scale. So I built a system that does.

The pipeline streams filings from EDGAR (10-K, 10-Q, 8-K, Form 4), extracts claims using FinBERT and SpaCy NER, embeds them, compares them against historical claims, and sends candidate contradictions to an LLM agent that decides whether the contradiction is real, how severe it is, and whether insiders traded around the time the language changed. The whole thing runs as a streaming pipeline with no manual triggers or batch jobs.

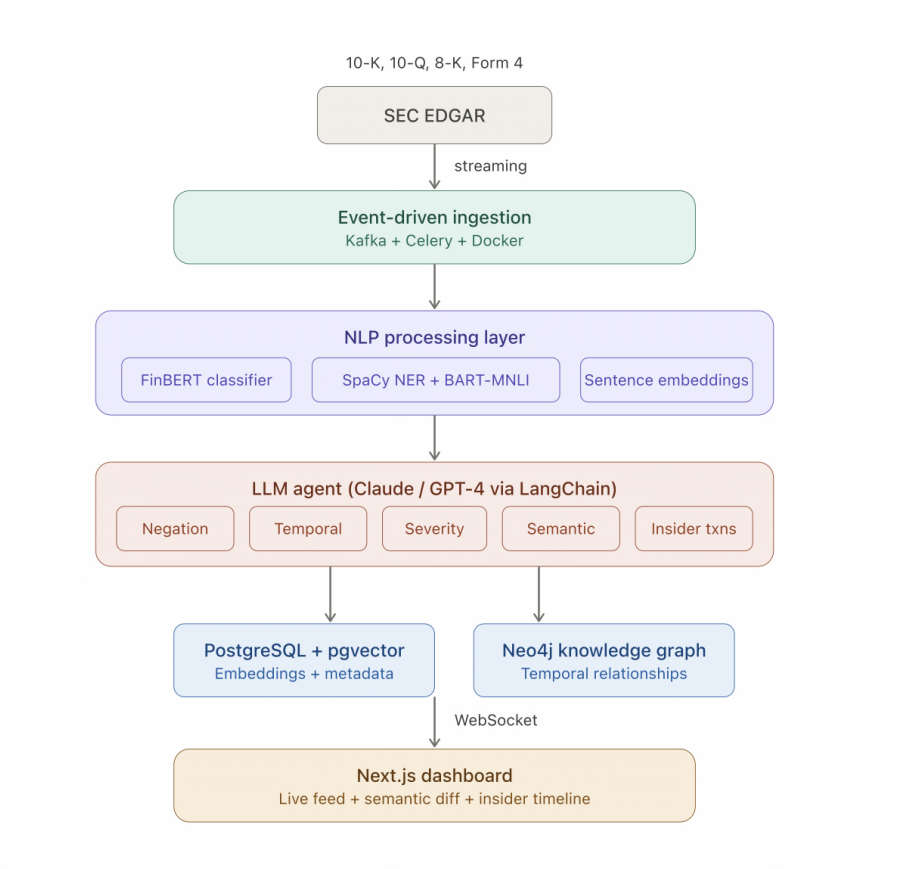

Figure 1: System architecture: EDGAR ingestion through agent analysis to dashboard.

Models and Frameworks

The contradiction detection pipeline runs five custom tools in a fixed orchestration order, with an LLM called only at the end for narrative reasoning:

1. **Negation detection** — a BART-MNLI zero-shot NLI classifier that scores whether two claims entail, contradict, or are neutral. This is the first filter; if NLI confidence is below threshold, the pair gets dropped before it ever hits the LLM.
2. **Temporal reasoning** — checks filing dates, determines which claim came first, calculates the time gap, and flags whether the chronological order is valid for comparison.
3. **Insider transaction lookup** — queries Form 4 data to check if any insiders bought or sold shares between the two filing dates. A CFO selling stock the week before the company quietly changes risk language is a different signal than a routine language update.
4. **Severity scoring** — combines the NLI score, cosine similarity, time gap, and insider activity into a composite severity rating (critical/high/medium/low).
5. **Semantic comparison** — uses sentence-transformer embeddings stored in PostgreSQL with pgvector to find the most similar historical claims for a given new claim.

The tools run in a fixed order after candidate retrieval: semantic comparison first to find high-similarity historical claims, then NLI to check for contradiction, temporal check to validate the date ordering, insider lookup (only if NLI clears the contradiction threshold, which saves unnecessary DB I/O), and finally severity scoring. After all tools have run, their outputs get bundled into a tool digest and passed to an LLM (Ollama running Mistral/TinyLlama locally) that generates a natural language reasoning trace explaining the final decision. This reasoning gets stored alongside the contradiction for auditability. The architecture is designed so you could swap in Claude or GPT-4 via API keys for the reasoning step, but the default runs fully local.

```
contradiction-detector-1  | 2026-04-21 03:16:27,423 [src.nli_scorer] INFO: NLI model loaded
Batches: 100%|██████████| 1/1 [00:00<00:00,  2.79it/s]
contradiction-detector-1  | 2026-04-21 03:16:27,783 [src.agent] INFO: [AGENT_TOOL] check_negation -> {"contradi
ction": 0.8053545951843262, "entailment": 0.012619832530617714, "neutral": 0.18202553689479828, "predicted_labe
l": "contradiction"}
contradiction-detector-1  | 2026-04-21 03:16:27,783 [src.agent] INFO: [AGENT_TOOL] temporal_check -> {"time_gap
_days": 6, "b_after_a": true, "valid_order_for_analysis": true, "note": "claim_b is newer than claim_a"}
contradiction-detector-1  | 2026-04-21 03:16:27,785 [src.agent] INFO: [AGENT_TOOL] get_insider_context -> {"win
dow": "2026-03-05 .. 2026-03-11", "transaction_count": 0, "sell_count": 0, "total_sell_value": 0.0, "large_insi
der_sales": false, "summary": "No insider transactions in window"}
contradiction-detector-1  | 2026-04-21 03:16:27,785 [src.agent] INFO: [AGENT_TOOL] severity_score -> {"severity
": "high", "insider_escalation": false, "factors": {"nli_contradiction": 0.8053545951843262, "similarity": 0.52
16092182164928, "time_gap_days": 6}}
contradiction-detector-1  | 2026-04-21 03:16:33,612 [httpx] INFO: HTTP Request: POST http://ollama:11434/api/ge
nerate "HTTP/1.1 200 OK"
contradiction-detector-1  | 2026-04-21 03:16:33,613 [shared.llm] INFO: Generated reasoning (1006 chars)
contradiction-detector-1  | 2026-04-21 03:16:33,613 [src.agent] INFO: [AGENT] decision {"is_contradiction": tru
e, "severity": "high", "claim_a_id": 292, "claim_b_id": 273}
contradiction-detector-1  | 2026-04-21 03:16:33,615 [src.detector] INFO:   Contradiction stored (severity=high,
 NLI=0.81): claims 292 vs 273
contradiction-detector-1  | 2026-04-21 03:16:33,615 [src.detector] INFO: Found 1 contradictions for filing 33
contradiction-detector-1  | 2026-04-21 03:16:33,615 [src.detector] INFO: Processed filing 33: 1 contradictions
```

Figure 2: Agent tool calls in action: `check_negation`, `temporal_check`, `get_insider_context`, and `severity_score` firing sequentially on a Costco filing pair.

On the NLP side, I fine-tuned FinBERT on roughly 5,000 labeled sentences for financial claim classification (factual, opinion, guidance, risk). Each claim also gets tagged with a topic (revenue, profitability, growth, regulatory) via BART-MNLI zero-shot classification, and SpaCy NER extracts entities, dates, and monetary values. All of this metadata feeds into the structured search layer so you can query things like "show me all revenue-related contradictions for tech companies in Q1."

The storage layer is split: PostgreSQL with pgvector holds the embeddings, structured claim metadata, and insider transaction records; Neo4j holds a knowledge graph where filings, claims, companies, and contradictions are nodes connected by typed edges (CONTAINS, ABOUT, CONTRADICTS, FILED, MADE). Insider transactions live in Postgres rather than the graph, since the agent tool queries them relationally by date window. The graph shines for structural queries — "what did Apple say about supply chain risk before and after this date" is a traversal, not a SQL join nightmare.

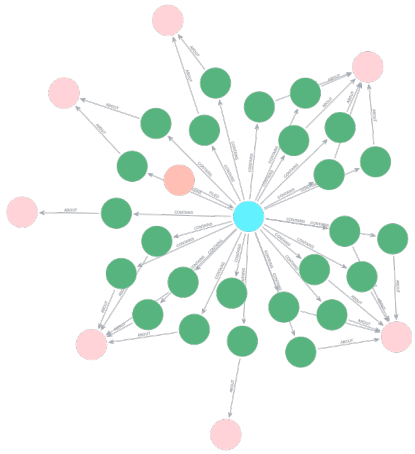

Figure 3: Neo4j knowledge graph: a filing node (blue) at center with CONTAINS edges to claim nodes (green) and ABOUT edges to company/topic nodes (pink).

The ingestion layer uses Kafka for event-driven processing and Celery for task distribution, all containerized with Docker Compose. The frontend is a Next.js dashboard with a live WebSocket feed that pushes new contradictions as they're detected.

Where Things Broke and How I Debugged It

The NLI model was way too aggressive. Early on, I was getting hundreds of "contradictions" per company and I thought the pipeline was working great — look at all these findings! Then I actually read them. A company saying "revenue grew 5%" in Q1 and "revenue grew 7%" in Q2 was getting flagged as a contradiction. Technically the NLI model isn't wrong — the two statements do say different things — but it's an update, not a contradiction. My first attempt at fixing this was adding a similarity threshold (only flag pairs above 0.85 cosine similarity), thinking that would filter out unrelated claims and leave the real contradictions. It did the opposite: it kept more boilerplate and dropped the interesting cases. The actual fix required stepping back and asking what makes a contradiction meaningful in a financial context. That's when I added the temporal reasoning tool. Now the pipeline checks: is claim B newer? Is this a topic that naturally updates over time (revenue figures, headcount)? If yes, it gets classified as an update, not a contradiction. Getting the topic taxonomy right took a few iterations — I started with 20+ categories and collapsed them down to the ones that actually affected the update-vs-contradiction decision.

Embedding similarity was a red herring for contradiction detection. I assumed contradictory claims would have high cosine similarity (same topic) but opposite sentiment, so I built the initial candidate retrieval around a high-similarity + sentiment-mismatch filter. The results were almost entirely boilerplate — legal disclaimers and risk factors that get copy-pasted between filings with minor wording changes. Actual contradictions tend to land in the 0.5–0.7 similarity range because the company is saying something substantively different, not just rewording the same sentence. I spent a while trying to tune the similarity threshold (0.7? 0.6? 0.55?) before realizing I was optimizing the wrong signal entirely. The fix was to drop similarity as a precision signal and use it only for recall — cast a wide net with a low threshold, then let the NLI model do the actual contradiction scoring. Severity scoring now weights NLI confidence much higher than cosine similarity. This feels obvious in retrospect, but I burned about a week on similarity tuning before I pulled the logs and actually looked at which pairs were getting through.

Form 4 data integration was messier than expected. SEC EDGAR returns Form 4 data in XML, and the formatting is inconsistent across filers. Some insiders file amendments that partially overlap with the original filing. I initially tried to parse these with a generic XML-to-dict conversion and immediately got duplicate transactions, missing fields, and dates in three different formats. The debugging was tedious — I'd pull a batch of Form 4s for a company, parse them, and manually cross-reference against the SEC website to see which transactions were real. I ended up writing specific deduplication rules (match on reporter CIK + transaction date + share count) and a date normalizer. Not glamorous work, but the insider context turned out to be one of the most valuable signals in the system. A company changing its risk language is one thing. A company changing its risk language the week after the CFO sold $2M in stock is a completely different conversation.

The LLM was over-reasoning and the output was unreliable. Two separate problems that felt like one. First, early versions of the Ollama reasoning step would write 500-token paragraph justifications for obvious cases (0.95 NLI score, 3-day gap — just say it's a contradiction). Second, the free-text output format was breaking my downstream parsing constantly. I was using regex to extract the severity and decision from the LLM's response, and every time the model phrased something slightly differently, the regex would miss it. The structured outputs fix (forcing JSON with a defined schema: is_contradiction, severity, reasoning) solved the parsing problem immediately. For the verbosity problem, I tuned the system prompt to encourage brevity proportional to ambiguity and added a token budget hint. Obvious cases now get a one-liner. Edge cases still get detailed

reasoning. This also made the local Ollama model (which is already constrained on quality) focus its limited capacity on the cases that actually needed it.

The Dashboard

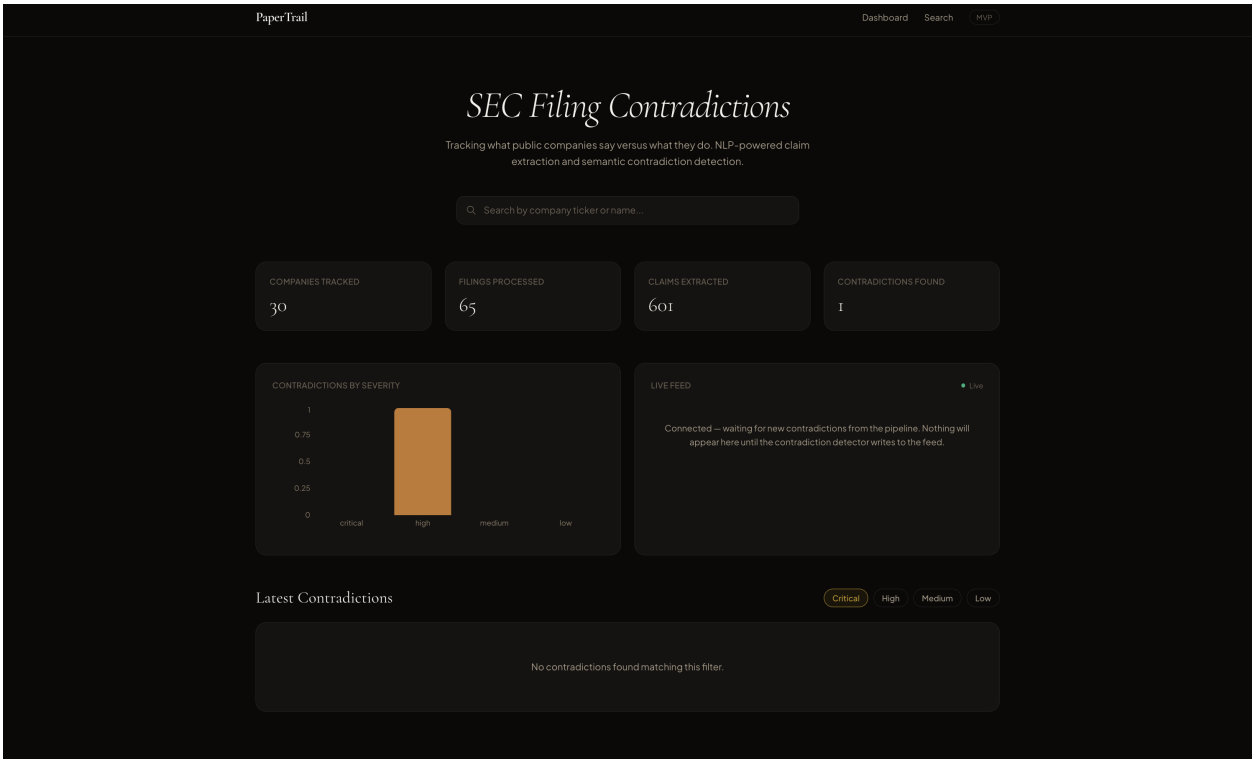

Figure 4: Main dashboard: 30 companies tracked, 65 filings processed, 601 claims extracted. Severity distribution chart and live WebSocket feed on the right.

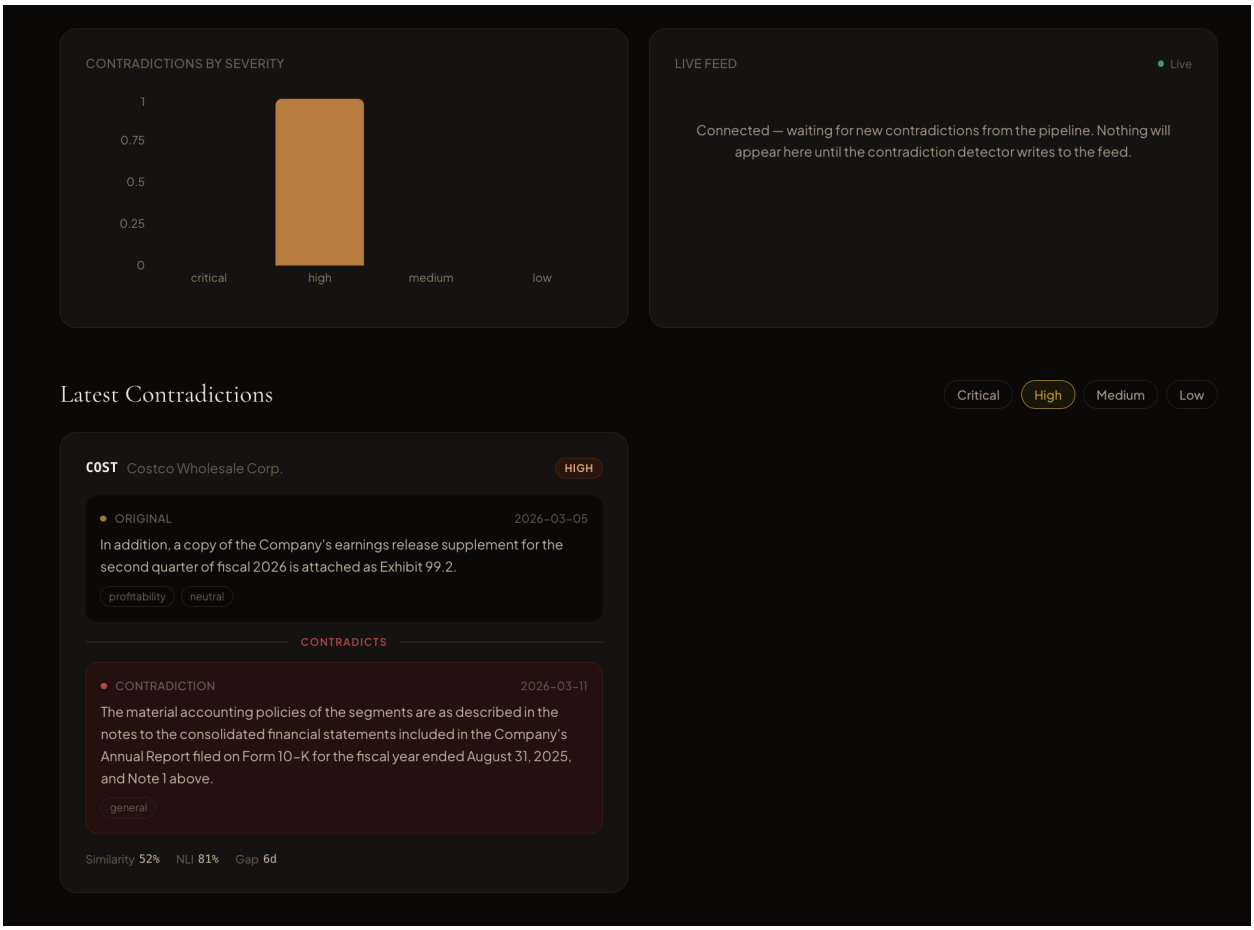

Figure 5: Contradiction detail view for Costco (COST): original claim from 2026-03-05 vs. contradicting claim from 2026-03-11, with similarity (52%), NLI score (81%), and time gap (6 days).

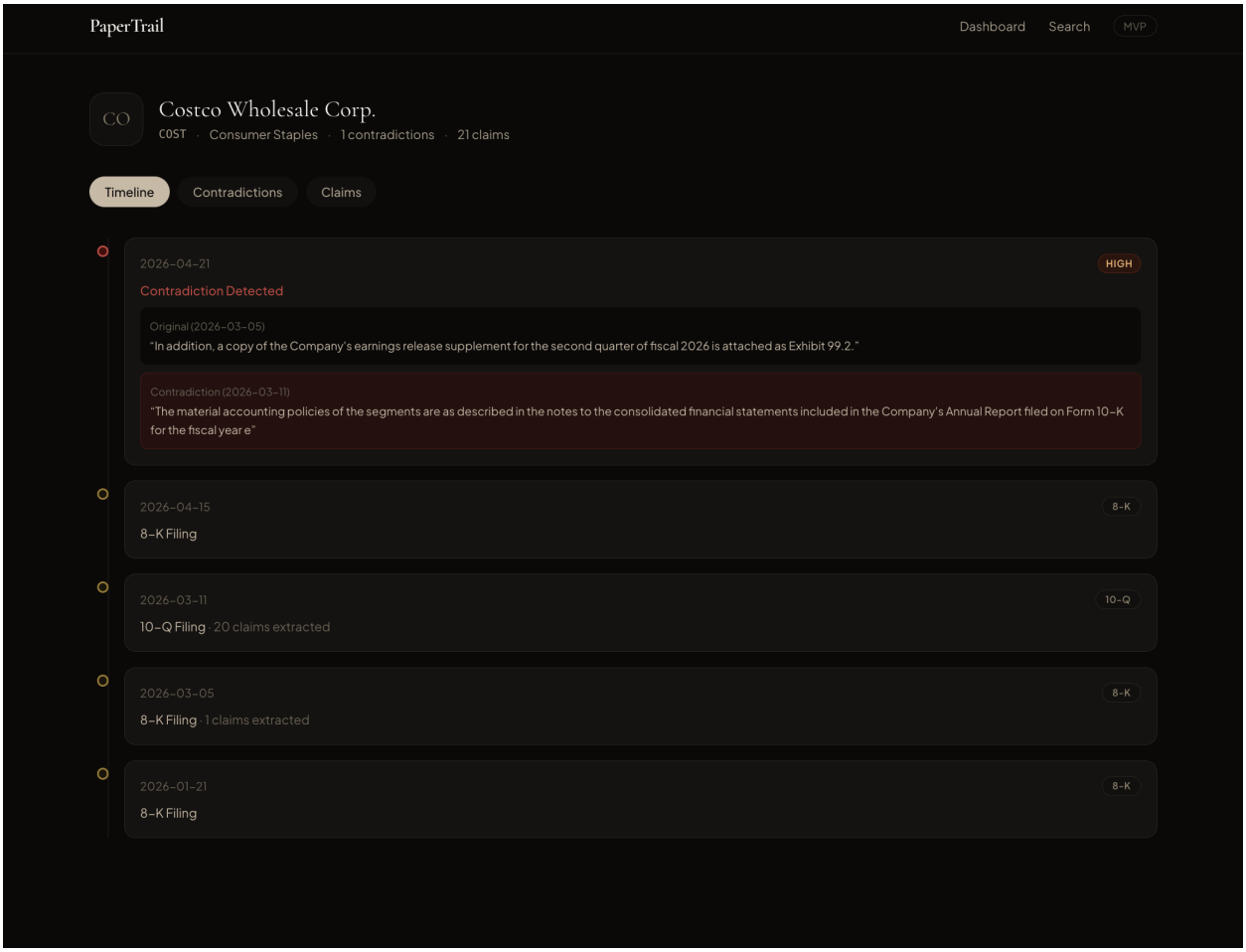

Figure 6: Company timeline: chronological filing history with contradiction events highlighted in context alongside 8-K and 10-Q filings.

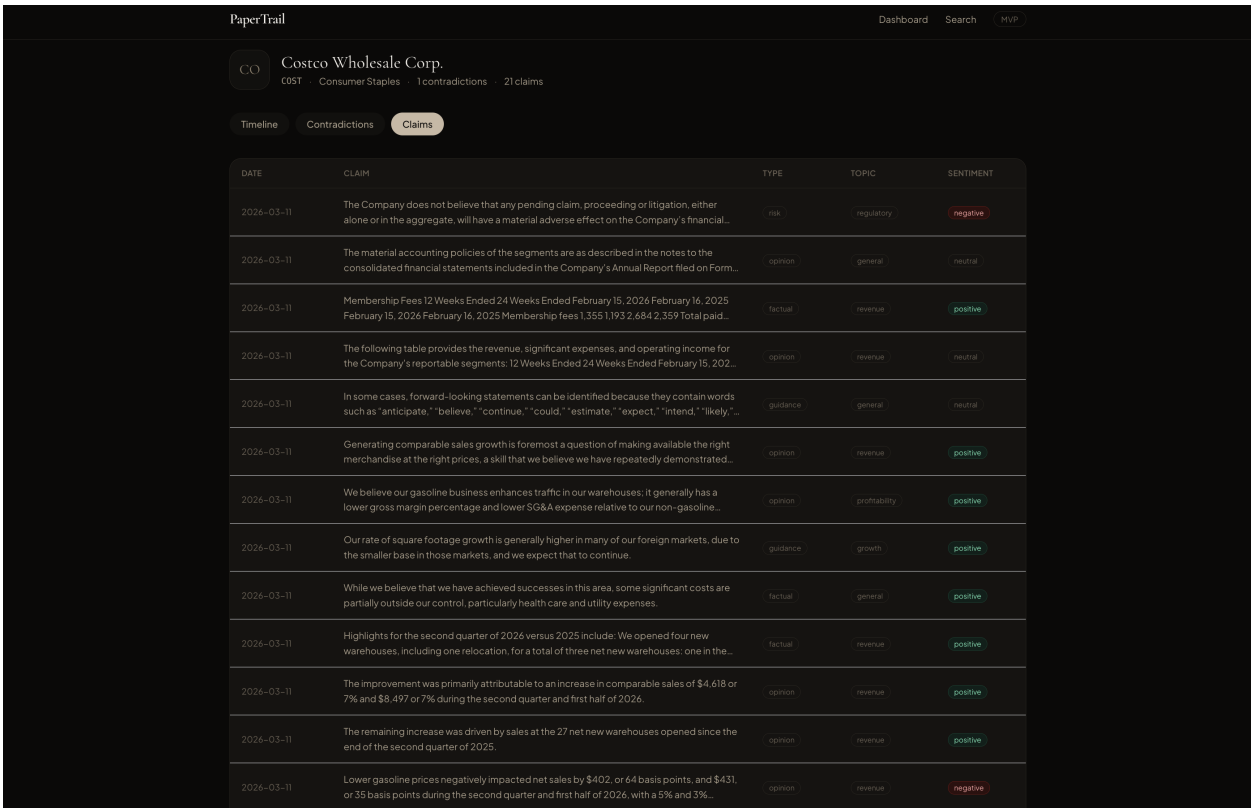

Figure 7: Extracted claims table: each claim classified by type (factual, opinion, guidance, risk), topic (revenue, profitability, growth), and sentiment.

GET /api/v1/contradictions/latest Get Latest Contradictions

Latest contradictions across all S&P 500 companies.

Parameters

Cancel

Name	Description
severity string | (string | null) (query)	high
limit integer (query)	20 maximum: 100

Execute Clear

Responses

Curl

```
curl -X 'GET' \
  'http://localhost:8000/api/v1/contradictions/latest?severity=high&limit=20' \
  -H 'accept: application/json'
```

Request URL

```
http://localhost:8000/api/v1/contradictions/latest?severity=high&limit=20
```

Server response

Code	Details
200	Response body


```
[
  {
    "id": 1,
    "company_ticker": "COST",
    "company_name": "Costco Wholesale Corp.",
    "claim_a": {
      "id": 292,
      "filing_id": 33,
      "claim_text": "In addition, a copy of the Company's earnings release supplement for the second quarter of fiscal 2026 is attached as Exhibit 99.2.",
      "claim_type": "opinion",
      "topic": "profitability",
      "sentiment": "neutral",
      "confidence": 0.9476333856582642,
      "entities": {
        "dates": [
          "the second quarter of fiscal 2026"
        ],
        "money": [],
        "persons": [],
        "percentages": [],
        "organizations": [
          "Company"
        ]
      },
      "temporal_ref": null
```


Download

Response headers

```
content-length: 1814
content-type: application/json
date: Tue,21 Apr 2026 03:04:45 GMT
server: uvicorn
```

Figure 8: FastAPI endpoint returning structured contradiction data with full claim metadata, entity extraction, and confidence scores.

What I'd Do Differently

Start with a smaller, denser dataset. I initially tried to track 100+ S&P 500 companies simultaneously to stress-test the pipeline. In hindsight, I should have started with 5–10 companies with known historical contradictions (there are SEC enforcement cases you can use as ground truth) and validated the agent's accuracy before scaling. The pipeline engineering worked fine at scale, but I didn't have a good evaluation framework early enough to know whether the contradictions it was finding were actually interesting.

Use structured outputs from the LLM from day one. I started with free-text agent responses and parsed them with regex. That broke constantly. Switching to structured JSON outputs with a defined schema (is_contradiction, severity, reasoning, claim_ids) made everything downstream dramatically more reliable. Should have done it from the start.

Build the graph schema before the relational schema. I added Neo4j midway through the project because temporal queries were getting painful in SQL. If I were rebuilding, I'd design the graph schema first and use PostgreSQL only for the vector search and bulk storage. The graph naturally models the relationships I care about (company → filing → claim → contradicts → claim), and trying to retrofit that onto foreign keys was a waste of time.

Use a frontier model for the reasoning step. The system currently runs Ollama with TinyLlama/Mistral locally for the narrative reasoning, which keeps everything self-contained and free but limits the quality of the agent's explanations on ambiguous cases. For production, I'd route edge cases (moderate NLI scores, mixed signals from tools) to Claude or GPT-4 via API for richer reasoning, and keep the local model for obvious high-confidence decisions. The architecture already supports this swap — it's just an API key and a config change — but I haven't benchmarked the quality difference yet.

GitHub: https://github.com/ketakiii3/papertrail